# Release note: Recommendation Engine Phase1

## Description:

Phase 1 contains two segments, nonpersonalised and similar items. Both overviews are given below with the api integration.

Please add the credentials that you'll receive from the developer in the Authorization panel of **POSTMAN** and select Digest Auth and then hit the API.

## Git Repository:

https://github.com/pb-admin1/Recommendation_Engine/tree/Recommendation_phase_1

## Nonpersonalised Overview:

This module provides a robust, nonpersonalized content recommendation engine for media or entertainment platforms. It is designed to:

Load, clean, and process item, user, and feedback datasets from CSV files.

Validate users and profiles, including handling guest and child profiles with appropriate restrictions.

Filter genres for child safety, ensuring compliance with strict content policies.

Generate recommendations based on recency (latest items) or popularity (weighted user interactions), with progressive fallback mechanisms to maximize result availability.

Support both adult and child profiles, with strict genre filtering and fallback logic for children.

Log all major operations for traceability and debugging.

## Data Loading and Cleaning

**load_and_clean_data()**

Purpose: Loads all datasets, applies column renaming, date parsing, duplicate removal, and validates data integrity.

Details:

- Checks for file existence and raises errors if missing.
- Reads CSVs into DataFrames.
- Renames columns for consistency (_rename_columns).

Parses date columns and merges earliest interaction dates.
Removes items with invalid release dates, but retains items without feedback.
Validates that datasets are not empty after cleaning.
Logs all steps and outcomes for traceability.

_rename_columns(), _fix_dates(), _remove_duplicates(), _validate_data()
Purpose: Internal helpers to standardize column names, parse dates, remove duplicates, and ensure data integrity.
Details:
_rename_columns: Ensures all DataFrames use a consistent schema; converts item_id to Int64 for merging.
_fix_dates: Converts date columns to datetime, handling errors gracefully.
_remove_duplicates: Removes duplicate items by item_id.
_validate_data: Raises errors if required DataFrames are empty after cleaning.

Filtering Logic

_apply_filters(df, country, genre, language, categories, content_type)
Purpose: Applies a set of optional filters to a DataFrame.
Parameters:
df: DataFrame to filter.
country, genre, language, categories, content_type: Optional string filters.
Returns: Filtered DataFrame.
Details:
Each filter is applied only if the parameter is provided and the column exists.
Uses caseinsensitive substring matching for flexibility.
Designed for extensibility (add more filters as needed).

### *User and Profile Validation*

validate_user_and_profile(user_id, profile_id, profile_type=None)
Purpose: Validates a user and profile, determines if the user is a guest or child, and returns allowed genres and status messages.
**Parameters**:
user_id: User UUID.
profile_id: Profile ID.
profile_type: Optional override for profile type.
**Returns**:
Dictionary with keys: is_valid, is_guest, is_child, profile_type, allowed_genres, message.
**Details**:
Handles guest users and unknown users with fallback logic.
Allows manual override of profile type, with validation against allowed types.

For child profiles, restricts allowed genres to kid_safe_genres.
Logs all validation steps and errors.
Returns detailed status and error messages for API consumers.
Validation/Fallback:
If user or profile not found, treats as guest with optional profile type override.
Invalid profile types default to Adult.

### *Genre Filtering and Child Safety*

normalize_genres(genre_string)
Purpose: Normalizes a commaseparated genre string to uppercase, removes duplicates, and cleans formatting.
Parameters:
genre_string: Commaseparated string of genres (any case).
Returns:
Normalized commaseparated string of uppercase genres.
Details:
Ensures consistent genre handling across the system.

filter_genres_for_kids(genre_string)
Purpose: Filters a commaseparated genre string to only include kidsafe genres.
Parameters:
genre_string: Commaseparated string of genres.
Returns:
Filtered commaseparated string of kidsafe genres.
Details:
Converts genres to uppercase and strips whitespace.
Returns empty string if no kidsafe genres are found.
Used in both user validation and recommendation filtering.

_filter_for_child_safety(df)
Purpose: Strictly filters a DataFrame to only include items where all genres are kidsafe and at least one is explicitly kidfocused.
Parameters:
df: DataFrame to filter.
Returns:
Filtered DataFrame.
Details:
Excludes items with NC (Not Classified) or missing genres.
Requires all genres to be in kid_safe_genres and at least one to be in a set of explicitly kidfocused genres.
Ensures compliance with strict child safety requirements.

Genre Discovery

get_available_genres_for_profile(is_child=False)
Purpose: Returns a sorted list of all genres present in the items dataset.
Parameters:
  is_child: Currently unused; all users see all genres in the UI.
Returns:
  List of unique genres (uppercase).
Details:
  Aggregates genres from all items, splitting and normalizing.
  UI can use this to populate genre selection widgets.

The API layer in app.py exposes endpoints for nonpersonalized content recommendations, user/profile validation, and genre/category discovery. It acts as the interface between client applications (such as web/mobile frontends) and the backend recommendation logic implemented in the NonPersonalizedRecommendationSystem class.

## **Similar-Items Overview**:

API:
https://recommendation.pb-online.co.in/api/similar-items?item_id=10631&age=6&country=India&top_k=10

Enhanced API for finding similar items using semantic embeddings and efficient similarity search.

Features

Fast Similarity Search: Uses FAISS for efficient nearest neighbor search
Semantic Embeddings: Powered by allMiniLML6v2 model for highquality embeddings
Caching: Intelligent caching of embeddings to avoid recomputation
Filtering: Support for multiple filters (country, language, categories, etc.)
RESTful API: Clean REST endpoints with comprehensive error handling

API Endpoints

Health Check

GET /health

Returns API health status and initialization state.

Get Item Details

GET /api/item/{item_id}

Get detailed information about a specific item.

Get Similar Items

GET https://recommendation.pb-online.co.in/api/similar-items?item_id=15377&top_k=10

Parameters:
top_k (optional): Number of similar items to return (default: 10)
country (optional): Filter by country
language (optional): Filter by language
categories (optional): Filter by categories (Movies, Shows, etc.)
content_type (optional): Filter by content type
genres (optional): Filter by genres

Non personalised overview:

## Latest API:

https://recommendation.pb-online.co.in/api/recommendations/latest?age=6&country=India&top_n=10&language=TELUGU&user_id=test&profile_id=test

Returns the latest items, prioritizing those that match the requested genres, with strict child safety filtering as needed.
Request Parameters:
`top_n` (int, optional, query or JSON): Number of items to return (default: 10).
`country`, `genre`, `language`, `categories`, `content_type` (string, optional): Filters for recommendations.
`user_id`, `profile_id`, `profile_type` (string, optional): For user validation and child safety.
Validation:
Parameters are validated for type and presence if required.
Genres are filtered for child safety if needed.
Response (JSON):
List of recommended items (each as a dict with item metadata).
May include additional metadata (e.g., total count, applied filters).
Status Codes:
`200 OK`: Recommendations returned.

`400 Bad Request`: Invalid parameters.
`500 Internal Server Error`: Backend error.
Authentication:
None by default.
Backend Logic:
Calls `get_latest_items()` in the recommendation system.
Notes:
Applies strict child safety filtering for child profiles or kidrelated genres.
Includes fallback logic to maximize result availability.

## Trending API:

https://recommendation.pb-online.co.in/api/recommendations/trending?age=16&country=India&top_n=10&gravity=0.5&user_id=test&profile_id=test

Returns trending items using a timedecay algorithm, with genre and child safety filtering.
Request Parameters:
`gravity` (float, optional): Decay parameter (default: 0.5).
`top_n` (int, optional): Number of items to return (default: 10).
`country`, `genre`, `language`, `categories`, `content_type` (string, optional): Filters.
`user_id`, `profile_id`, `profile_type` (string, optional): For user validation and child safety.
Validation:
Parameters are validated for type and presence if required.
Response (JSON):
List of trending items (each as a dict with item metadata and trending score).
Status Codes:
`200 OK`: Recommendations returned.
`400 Bad Request`: Invalid parameters.
`500 Internal Server Error`: Backend error.
Authentication:
None by default.
Backend Logic:
Calls `get_trending_items()` in the recommendation system.
Notes:
Trending score combines interaction recency, frequency, and content recency.
Child safety filtering is applied as needed.

# Environment variables documentation for sand box deployment

Requirements.txt file:

```
blinker==1.9.0
boto3==1.39.13
botocore==1.39.13
certifi==2025.7.14
charset-normalizer==3.4.2
click==8.1.8
cmake==4.0.3
contourpy==1.3.0
cycler==0.12.1
faiss-cpu==1.7.4
filelock==3.18.0
Flask==2.3.3
flask-cors==6.0.1
fonttools==4.59.0
fsspec==2025.7.0
huggingface-hub==0.19.4
idna==3.10
importlib_metadata==8.7.0
importlib_resources==6.5.2
itsdangerous==2.2.0
Jinja2==3.1.6
jmespath==1.0.1
joblib==1.5.1
kiwisolver==1.4.7
lit==18.1.8
MarkupSafe==3.0.2
matplotlib==3.9.4
mod-wsgi==5.0.2
mpmath==1.3.0
networkx==3.2.1
```

```
nltk==3.9.1
numpy==1.24.3
nvidia-cublas-cu11==11.10.3.66
nvidia-cuda-cupti-cu11==11.7.101
nvidia-cuda-nvrtc-cu11==11.7.99
nvidia-cuda-runtime-cu11==11.7.99
nvidia-cudnn-cu11==8.5.0.96
nvidia-cufft-cu11==10.9.0.58
nvidia-curand-cu11==10.2.10.91
nvidia-cusolver-cu11==11.4.0.1
nvidia-cusparse-cu11==11.7.4.91
nvidia-nccl-cu11==2.14.3
nvidia-nvtx-cu11==11.7.91
packaging==25.0
pandas==2.0.3
pillow==11.3.0
pyparsing==3.2.3
python-dateutil==2.9.0.post0
python-dotenv==1.0.0
pytz==2025.2
PyYAML==6.0.2
regex==2024.11.6
requests==2.32.4
s3transfer==0.13.1
safetensors==0.5.3
scikit-learn==1.3.0
scipy==1.13.1
seaborn==0.13.2
sentence-transformers==2.3.1
sentencepiece==0.2.0
six==1.17.0
sympy==1.14.0
threadpoolctl==3.6.0
tokenizers==0.15.2
torch==2.0.1
tqdm==4.67.1
transformers==4.35.2
triton==2.0.0
typing_extensions==4.14.1
tzdata==2025.2
urllib3==1.26.20
Werkzeug==3.1.3
```

```
zipp==3.23.0
```

.env file:

```
S3_BUCKET = "sagemaker-studio-533267087662-kncjumq6vln"
IAM_ROLE_ARN = "arn:aws:iam::533267087662:role/role-assume-s3full"
S3_ITEMS_FOLDER = 'ITEMS_RECOMMENDATION_DATA/active_items.csv'
S3_USERS_FOLDER = 'USERS_RECOMMENDATION_DATA/users_recommendation_data.csv'
S3_FEEDBACK_FOLDER = 'FEEDBACK_RECOMMENDATION_DATA/feedback data for
recommendation.csv'
S3_ITEM_DATA_WITH_AGE_CERT = 'ITEMS_AGE_CERTIFICATE_RECOMMENDATION_DATA/show
certification data.csv'
```

Step1: Create a virtual environment.

Step2: Create an ARN role and attach that role in the .env file that is present in the root folder.

Step3: Install all dependencies from the requirements.txt file into the virtual environment.

Step4: Run this command in terminal after activating the virtual environment:

python run_data_pipeline.py

Or

python3 run_data_pipeline.py

Step5: after  running this command run:

python -m item-item.app

Or

python item-item/app.py

Check the health status of the api by this route `/health`

Check the api statistics by this route `` `/api/stats` ``

Check all three api's on these route:

`/api/recommendations/latest`,

`/api/recommendations/trending' ,

`/api/similar-items/{item_id}`

Cron jobs:

- Cron job for the run_data_pipeline.py file everyday at 2am.
- Cron job for the app.py file every day at 3am

Systemctl flask.service file to start the instance with item-item/app.py file so that the server starts whenever the instance is launched.

Add port 8080 in security groups in the ec2, add s3 permission on ec2.

Add DNS to the api